

Project: Desktop-Style OS Emulator

CSOPESY Term 1 Report

📅 Date

Author: 👤 Person

Lead Architect: Sean Marthy Arambulo

I. Video Walkthrough

Please insert a link to your hosted video demonstration below. The video should cover bootstrapping, process scheduling, and memory allocation.

Link: 📁 File

Key Timestamp Highlights:

- **00:00 - 00:30:** Bootstrapping Sequence and POST.
 - **00:31 - 01:45:** Kernel Initialization and System Services.
 - **01:46 - 03:00:** Simulation of CPU Scheduling (Round Robin/FCFS).
 - **03:01 - End:** Memory Management and Swapping to Backing Store.
-

II. Architectural Diagram

The OS design follows a **Layered Architecture** to ensure modularity and ease of debugging.

Layer	Component	Functionality
User Space	UI / CLI	Interaction via Command Line Interface or GUI.
System Services	Background Daemons	Networking, File Systems, and Logging.
Kernel	Core Manager	CPU Scheduling and Memory Management.
Hardware Abstraction	HAL / Drivers	Mapping software to physical resources.

III. Code Snippets: Kernel Lifecycle

The following implementation follows the standard execution sequence for an OS Emulator.

1. Main Entry Point

None

```
int main() {  
    // Hardware-level initialization  
    bootstrap();  
}
```

```
// Core kernel data structures (Process Table, Memory Table)
initializeKernel();

// Launch persistent background services
startSystemServices();

// User interaction loop
showWelcomeUI();
enterMainLoop();

// Graceful termination
shutdownAndCleanup();
}
```

2. Main Executive Loop

```
None
void enterMainLoop() {
    while (!shutdownInitiated) {
        // Continuously check for user input and hardware interrupts
        handleIO();

        // Dispatch next process based on scheduling algorithm
        cpuScheduler.dispatch();

        // Check for scarcity and initiate swapping if needed
        memoryManager.checkUtilization();
    }
}
```

IV. Design Discussion

This desktop-style OS was designed to provide a high-level abstraction of hardware resources while maintaining an efficient execution environment.

Core Philosophy

The design prioritizes the Layered Approach, treating the lowest layer as hardware and the highest as the User Interface. This ensures that changes to the UI do not compromise the stability of the kernel.

1. Bootstrapping and Initialization

The sequence begins with a Bootstrapping phase, which performs low-level setup like initializing the Hardware Abstraction Layer (HAL) and enabling the Floating Point Unit (FPU). This is followed by Kernel Initialization, where the process table and memory manager are established.

2. Process and CPU Management

The OS acts as a Resource Allocator, managing conflicting requests for CPU time. The emulator utilizes threads to simulate individual CPUs, allowing for the concurrent execution of user and system processes.

3. Memory Management Strategy

A Flat Memory Model is implemented, where memory is divided into predefined partitions.

- **Internal Fragmentation:** Handled by allocating the smallest possible "hole" for a process.
- **Virtual Memory & Swapping:** To provide the illusion of infinite memory, the system uses a **Backing Store** (SSD/HDD). When RAM is exhausted, less active processes are swapped out to allow new processes to execute.

4. Input/Output and Display

The UI is built as an **Event-Driven** system for the CLI, refreshing only upon user input. However, the backend remains **Real-Time**, continuously handling background tasks and updates even when the user is idle.